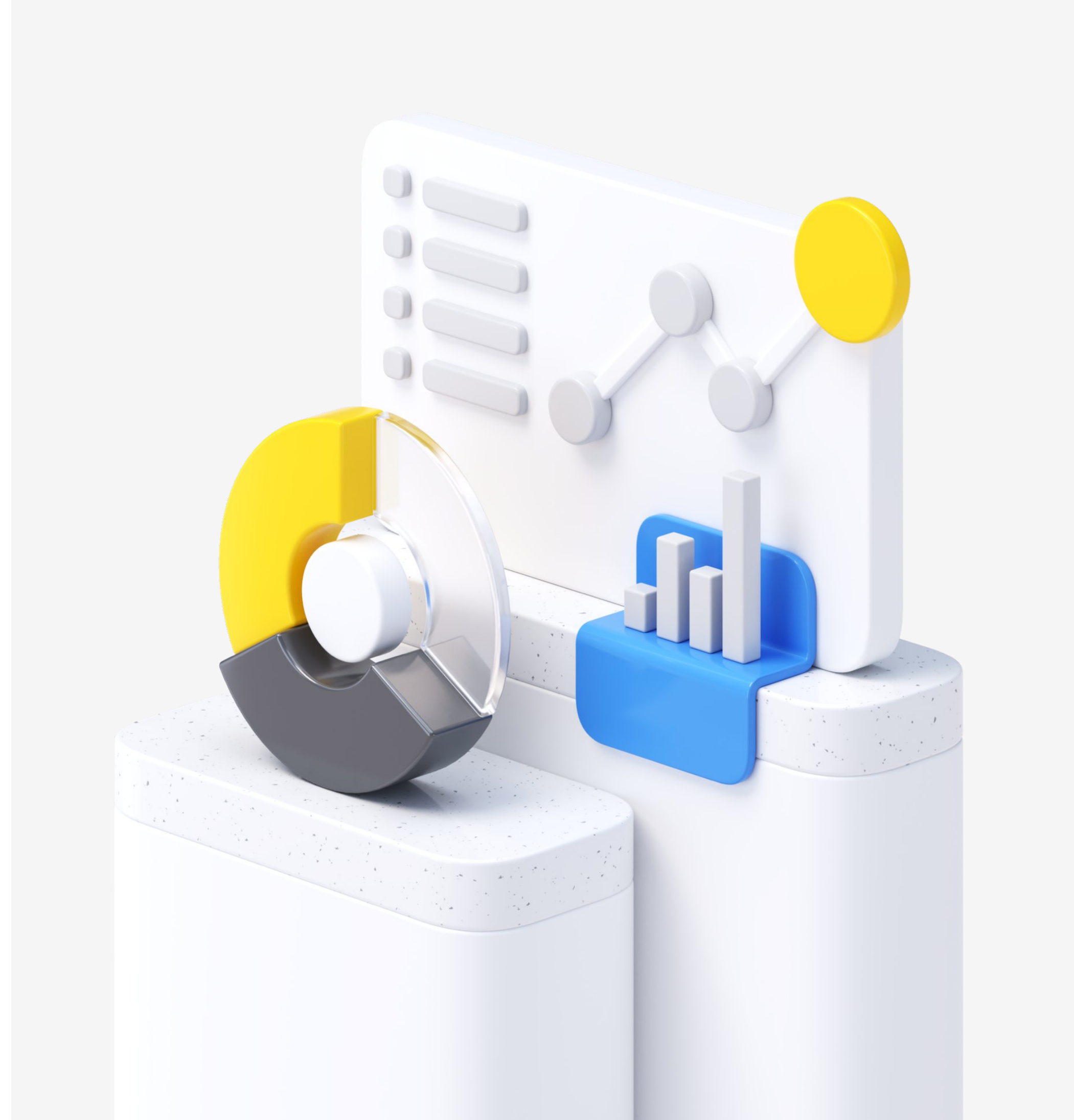




Лекция 3

Swift: структуры, классы, функции и замыкания

Анастасия, t.me/nasty_ru



План презентации

- **Функции**
- **Структуры и классы**
- **Замыкания**
- **Дополнительные функции**

План презентации

- **функции**
- Структуры и классы
- Замыкания
- Дополнительные функции

**Функции в Swift — это блоки кода,
которые выполняют определенную задачу**

функции Swift

- Способны принимать значения
- Способны отдавать значения
- Могут вызывать другие функции
- Из названия функции должна быть понятна ее задача
- Для имени функции используется lowerCamelCase

Функции Swift

Объявление:

- `func` — ключевое слово языка
- `funcName` — имя функции
- `parameters` — входные данные, параметры
- `returnType` — возвращаемый тип

```
func funcName(parameters) -> returnType {  
    // Тело функции  
}
```

Функции Swift

Вызов

```
func greetingFor(name: String) -> String {  
    let greeting = "Hello, \(name)!"  
    return greeting  
}
```

```
print(greetingFor(name: "world"))
```

Функции Swift

Вызов

```
func greetingFor(name: String) -> String {  
    "Hello, \(name)!"  
}
```

```
print(greetingFor(name: "world"))
```

Функции Swift

Типы

- Функции без параметров

```
func greetingForWorld() -> String {  
    "Hello, world!"  
}  
print(greetingForWorld())
```

Функции Swift

Типы

- Функции с несколькими входными параметрами

```
enum DayPart {  
    case morning  
    case day  
    case evening  
}
```

```
func greetingAllDayFor(name: String, dayPart: DayPart) -> String {  
    switch dayPart {  
    case .morning:  
        "Good morning, \(name)!"  
    case .day:  
        "Have a nice day, \(name)!"  
    case .evening:  
        "Good evening, \(name)!"  
    }  
}
```

```
print(greetingAllDayFor(  
    name: "world",  
    dayPart: .evening  
))
```

Функции Swift

Типы

- Функции, невозвращающие значение

```
func sayHelloToWorld() {  
    print("Hello, world!")  
}  
sayHelloToWorld()
```

Функции Swift

Типы

- Функции, возвращающие несколько значений

```
func minMax(array: [Int]) -> (min: Int, max: Int) {  
    let minValue = array.min() ?? array[0]  
    let maxValue = array.max() ?? array[0]  
    return (minValue, maxValue)  
}
```

```
minMax(array: [1,2,3,4,5,6])
```

Функции Swift

Типы

- Функции, возвращающие опционал

```
func sumOfMinAndMax(array: [Int]) -> Int? {  
    let minValue = array.min()  
    let maxValue = array.max()  
  
    guard  
        let minValue,  
        let maxValue  
    else {  
        return nil  
    }  
  
    return minValue + maxValue  
}
```

```
sumOfMinAndMax(array: [1,2,3,4,5,6])  
sumOfMinAndMax(array: [])
```

Функции Swift

Параметры

Каждый параметр функции имеет ярлык аргумента и имя параметра.

- Ярлык аргумента используется при вызове функции.
- Параметр в теле функции.

```
func function(argumentLabel parameterName: Int) {  
    parameterName + argumentLabel ❌ Cannot find 'argumentLabel' in scope  
}
```

```
function(argumentLabel: 0)
```

Функции Swift

Параметры

Также необязательно задавать для всех параметров ярлыки

```
func greeting(  
    for name: String,  
    dayPart: DayPart  
) -> String {  
    switch dayPart {  
    case .morning:  
        "Good morning, \(name)!"  
    case .day:  
        "Have a nice day, \(name)!"  
    case .evening:  
        "Good evening, \(name)!"  
    }  
}  
greeting(for: "world", dayPart: .day)
```

Функции Swift

Параметры

Пропуск ярлыков аргументов

```
func sumOfMinAndMax(_ array: [Int]) -> Int? {  
    let minValue = array.min()  
    let maxValue = array.max()  
  
    guard  
        let minValue,  
        let maxValue  
    else {  
        return nil  
    }  
  
    return minValue + maxValue  
}  
  
sumOfMinAndMax([1,2,3])
```

Функции Swift

Параметры

Для любого параметра можно использовать значение по умолчанию

```
func greeting(  
    for name: String,  
    dayPart: DayPart = .day  
) -> String {  
    switch dayPart {  
    case .morning:  
        "Good morning, \(name)!"  
    case .day:  
        "Have a nice day, \(name)!"  
    case .evening:  
        "Good evening, \(name)!"  
    }  
}  
  
greeting(for: "world", dayPart: .evening)  
greeting(for: "world")
```

ФУНКЦИИ

Области видимости

```
let globalName = 1

func usageOf(localName: Int) {
    print("\(localName + globalName)")
}

usageOf(localName: 2)
```

```
print("\(localName + globalName)")
```



Cannot find 'localName' in scope

План презентации

- **функции**
- **Структуры и классы**
- **Замыкания**
- **Дополнительные функции**

Структуры и классы — это универсальные и гибкие конструкции, которые становятся строительными блоками для кода программы

Структуры и классы

Общее

- Объявлять свойства для хранения значений
- Объявлять методы
- Объявлять инициализаторы
- Могут быть расширены (с помощью Extension)
- Могут соответствовать протоколам (обеспечение стандартной функциональности)

Структуры и классы

Общее

- Фактически создаем новый тип в Swift
- Для именования используем UpperCamelCase, чтобы соответствовать стандартам написания имен типов в Swift
- Но при это назначайте свойствам и методам имена в lowerCamelCase

Определение структур и классов

```
class Student {  
    var name: String = ""  
    var surname: String = ""  
    var averageScore: Double?  
}
```

```
struct University {  
    var name: String = ""  
    var adress: String?  
    var numberOfStudents: Int = 0  
}
```

Структуры и классы

Создание экземпляра

```
var student1 = Student()  
var university1 = University()
```

Структуры и классы

Доступ к свойствам

```
student1.name = "Настя"  
student1.averageScore = 4.3
```

```
university1.name = "МГТУ"  
university1.numberOfStudents = 1001
```

Структуры и классы

Поэлементный инициализатор

```
var university2 = University(  
    name: "University2",  
    adress: "Moscow",  
    numberOfStudents: 20  
)
```

Структуры и классы

Инициализатор

```
class Student {  
    var name: String = ""  
    var surname: String = ""  
    var averageScore: Double?  
  
    init(  
        name: String,  
        surname: String,  
        averageScore: Double? = nil  
    ) {  
        self.name = name  
        self.surname = surname  
        self.averageScore = averageScore  
    }  
  
    init() {}  
}
```

Определение структур и классов

```
class ClassName {  
    // определение класса  
}
```

Reference тип (ссылочный тип)

*При копировании копируется ссылка
на объект*

```
struct StructureName {  
    // определение структуры  
}
```

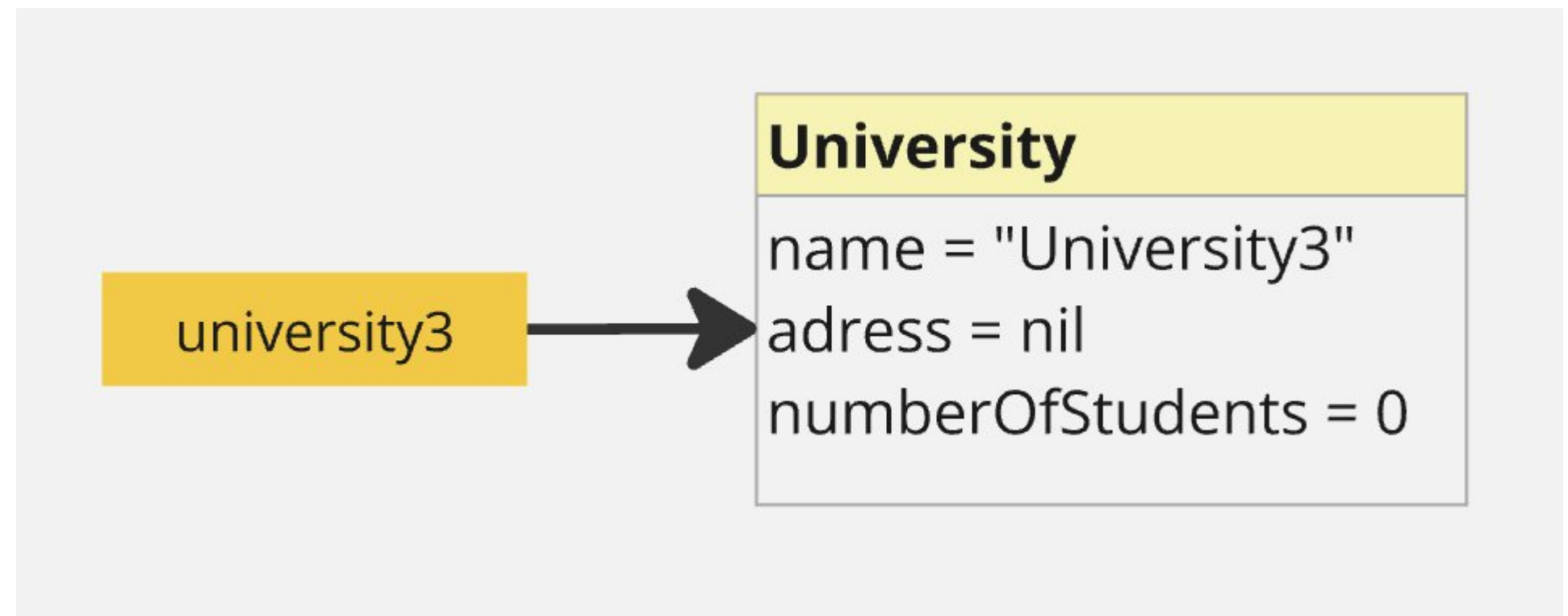
Value тип (тип-значение)

При копировании создается новый объект

Структура

Тип-значение

```
var university3 = University(  
    name: "University3"  
)
```

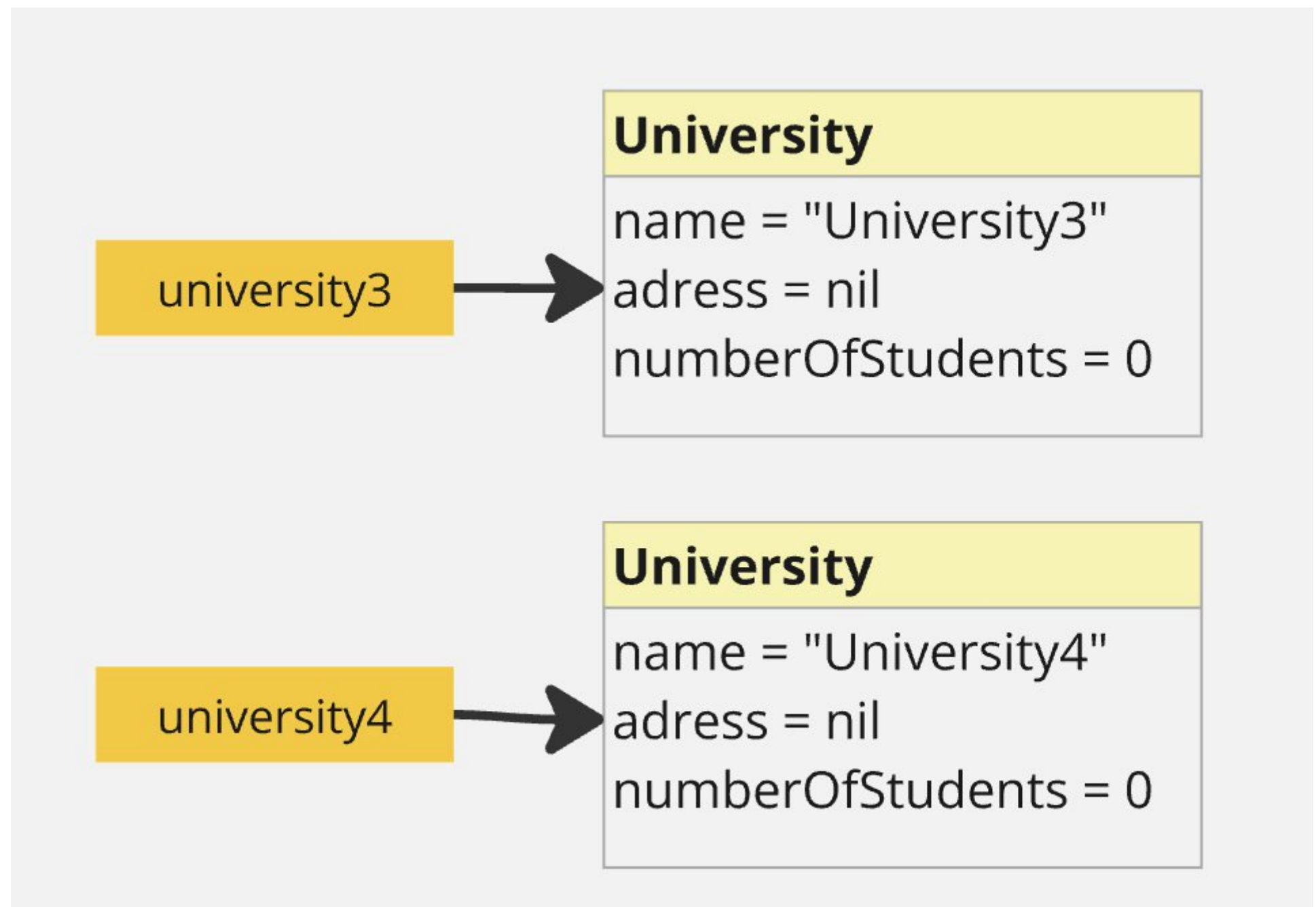


Структура

Тип-значение

```
var university4 = university3  
university4.name = "University4"
```

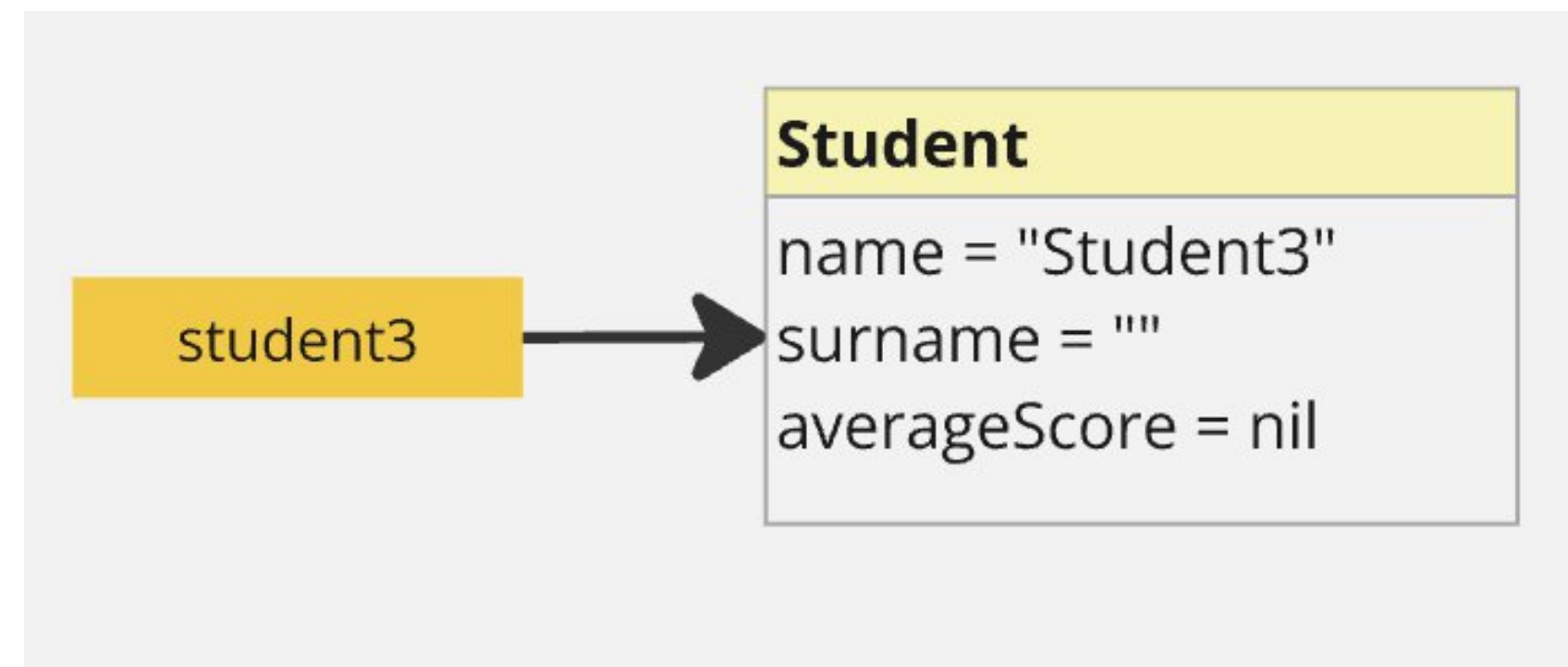
```
university3.name // University3  
university4.name // University4
```



Класс

Ссылочный тип

```
let student3 = Student()  
student3.name = "Student3"
```

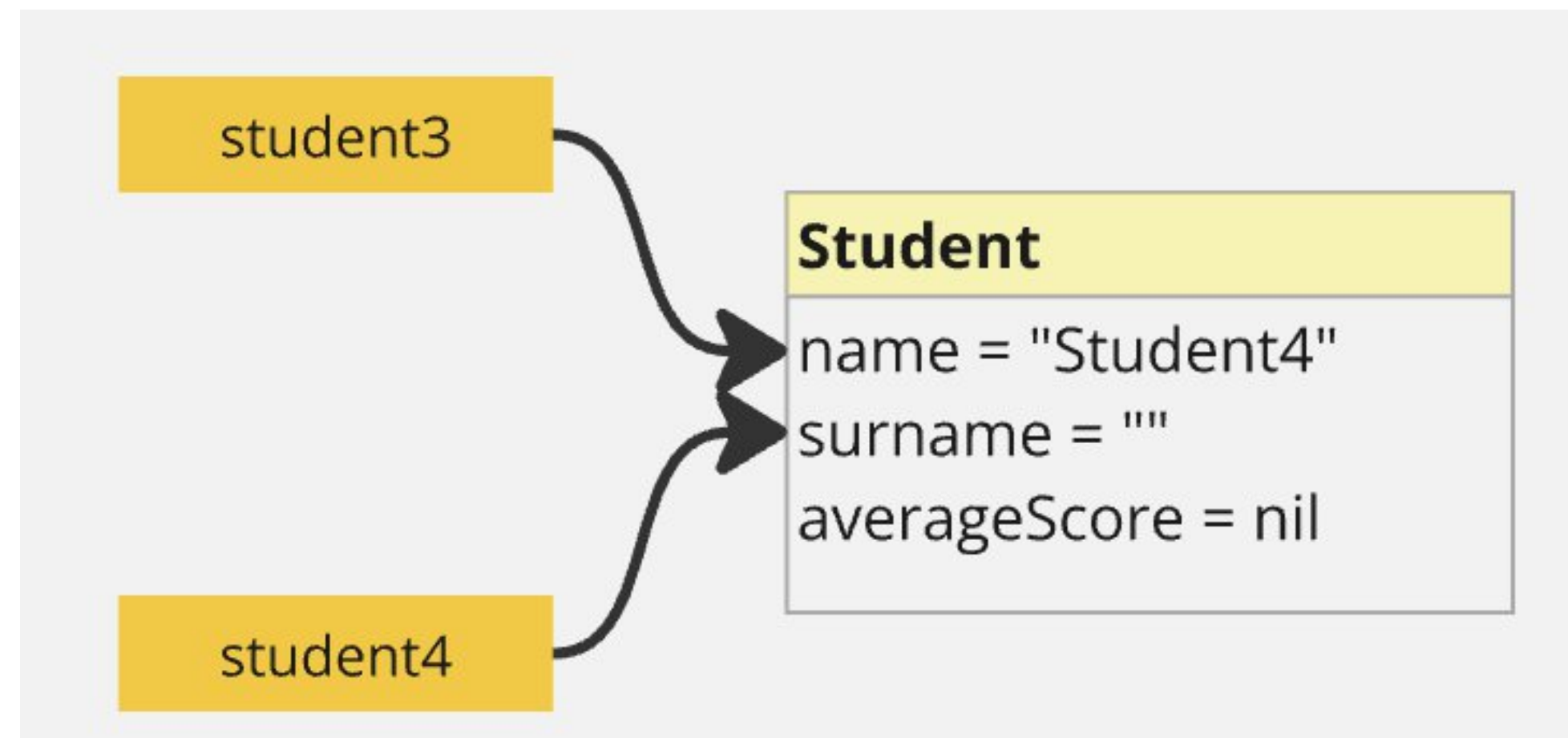


Класс

Ссылочный тип

```
let student4 = student3  
student4.name = "Student4"
```

```
student3.name // Student4  
student4.name // Student4
```



Структуры и классы

Структуры:

- Простые сущности
- Данные должны копироваться при передаче
- Объект не требует наследования

Классы:

- Объекты с поведением
- Объект должен передаваться по ссылке
- Общий изменяемый объект
- Требуется наследование
- Важен жизненный цикл объекта

План презентации

- Структуры и классы
- функции
- **Замыкания**
- Дополнительные функции

Калькулятор массивов

```
func multiplyByTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element * 2)  
    }  
    return modifiedArray  
}
```

```
func divideByTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element / 2)  
    }  
    return modifiedArray  
}
```

```
func sumWithTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element + 2)  
    }  
    return modifiedArray  
}
```

Калькулятор массивов

```
func multiplyByTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element * 2)  
    }  
    return modifiedArray  
}
```

```
func divideByTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element / 2)  
    }  
    return modifiedArray  
}
```

```
func sumWithTwo(array: [Double]) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(element + 2)  
    }  
    return modifiedArray  
}
```

Замыкания

```
func function()  
func function(param1: Int)  
func function(param1: Int) -> Int
```

Избавимся от всех именований:

```
() -> Void или () -> ()  
(Int) -> Void или (Int) -> ()  
(Int) -> Int
```

И получился тип данных, который и будет описывать наши замыкания

Замыкание aka Closure aka Lambda aka Анонимная функция — блоки с определенным функционалом, которые могут быть переданы и использованы в вашем коде.

Замыкание

```
let closure1: () -> () = {  
    print("closure")  
}  
let closure1Another = {  
    print("closure")  
}  
  
let closure2: (Int) -> () = { parameter in  
    print("closure with parameter: \"(parameter)\")  
}  
  
let closure3: (Int) -> Int = { parameter in  
    parameter * 2  
}  
let closure3Another = { (parameter: Int) -> Int in  
    parameter * 2  
}
```

Калькулятор массивов

```
func applyFor(array: [Double], action: (Double) -> Double) -> [Double] {  
    var modifiedArray: [Double] = []  
    for element in array {  
        modifiedArray.append(action(element))  
    }  
    return modifiedArray  
}  
  
let multiplyByTwo: (Double) -> Double = { argument in  
    argument * 2  
}  
  
let divideByTwo: (Double) -> Double = { argument in  
    argument / 2  
}  
  
let sumWithTwo: (Double) -> Double = { argument in  
    argument + 2  
}  
  
applyFor(array: [1,2,3], action: multiplyByTwo)
```

Замыкания

Избегающие

(отложенное поведение)

```
class ViewController {  
    let finishClosure: () -> ()
```

```
    init(finishClosure: () -> Void) {
```

```
        self.finishClosure = finishClosure
```

```
    }
```

```
    func finish() {
```

```
        finishClosure()
```

```
    }
```

```
}
```



Assigning non-escaping parameter 'finishClosure' to...

Замыкания

Избегающие

(отложенное поведение)

```
class ViewController {  
    let finishClosure: () -> ()  
  
    init(finishClosure: @escaping () -> Void) {  
        self.finishClosure = finishClosure  
    }  
  
    func finish() {  
        finishClosure()  
    }  
}
```

Замыкания

Это ссылочный тип

```
let closure = {  
    print("closure")  
}  
let closureCopy = closure // копируем ссылку на ТОЖЕ замыкание
```

Замыкания

Захват переменных

```
var counter = 0
```

```
let counterClosure1 = { // захват по ссылке  
  counter += 1  
}
```

```
let counterClosure2 = { [counter] in // создание копии  
  counter += 1  
}
```



Left side of mutating operator isn't mutable: 'counter' is an immutable capture

Замыкания

Захват переменных

```
1 var counter = 0
2
3 let counterClosure1 = { // захват по ссылке
4     print("From 1st closure \(counter)")
5 }
6
7 let counterClosure2 = { [counter] in // создание копии
8     print("From 2nd closure \(counter)")
9 }
10
11 counterClosure1()
12 counterClosure2()
13 counter = 4
14 counterClosure1()
15 counterClosure2()
```



```
From 1st closure 0
From 2nd closure 0
From 1st closure 4
From 2nd closure 0
```

План презентации

- Структуры и классы
- функции
- Замыкания
- **Дополнительные функции**

map

Преобразование элементов коллекции

Используется, когда нужно изменить каждый элемент массива или другого последовательного типа.

```
let numbers: [Double] = [1,2,3,4,5]

let multiplied = numbers.map { argument in
    multiplyByTwo(argument)
}
print(multiplied)

// [2.0, 4.0, 6.0, 8.0, 10.0]
```

map

```
let multiplied2 = numbers.map { multiplyByTwo($0) }  
print(multiplied2)  
  
// [2.0, 4.0, 6.0, 8.0, 10.0]
```

map

```
let multiplied3 = numbers.map(multiplyByTwo)
print(multiplied3)

// [2.0, 4.0, 6.0, 8.0, 10.0]
```

mapValues

Преобразование значений словаря

Аналог map, но работает с Dictionary, изменяя только значения.

```
let courses = [
    "Student1": 1,
    "Student2": 3,
    "Student3": 2
]
let nextYear = courses.mapValues { $0 + 1 }
print(nextYear)

// ["Student1": 2, "Student2": 4, "Student3": 3]
```

filter

Фильтрация элементов

Используется, чтобы выбрать только те элементы, которые соответствуют условию

```
let numbers = [10, 15, 22, 33, 40, 51]
let evenNumbers = numbers.filter { $0 % 2 == 0 }
print(evenNumbers)
```

```
// [10, 22, 40]
```

reduce

Агрегация значений

Используется для сворачивания коллекции в одно значение

```
let numbers = [1, 2, 3, 4, 5]
let sum = numbers.reduce(0) { $0 + $1 }
print(sum) // 15
```

```
let words = ["Swift", "is", "awesome"]
let sentence = words.reduce("") { $0 + " " + $1 }
print(sentence) // Swift is awesome
```

Как работает reduce?

0 или "" – начальное значение (первый аргумент функции).

\$0 – накопленный результат.

\$1 – текущий элемент.

flatMap

Убирает вложенные коллекции

Используется, когда есть массив массивов и нужно сделать его плоским.

```
let array = [[1, 2, 3], [4, 5], [6]]  
let flattened = array.flatMap { $0 }  
print(flattened) // [1, 2, 3, 4, 5, 6]
```

compactMap

Убирает nil

```
let optionalNumbers: [Int?] = [1, nil, 3, nil, 5]  
let nonNilNumbers = optionalNumbers.compactMap { $0 }  
print(nonNilNumbers) // [1, 3, 5]
```

zip

Объединение двух массивов

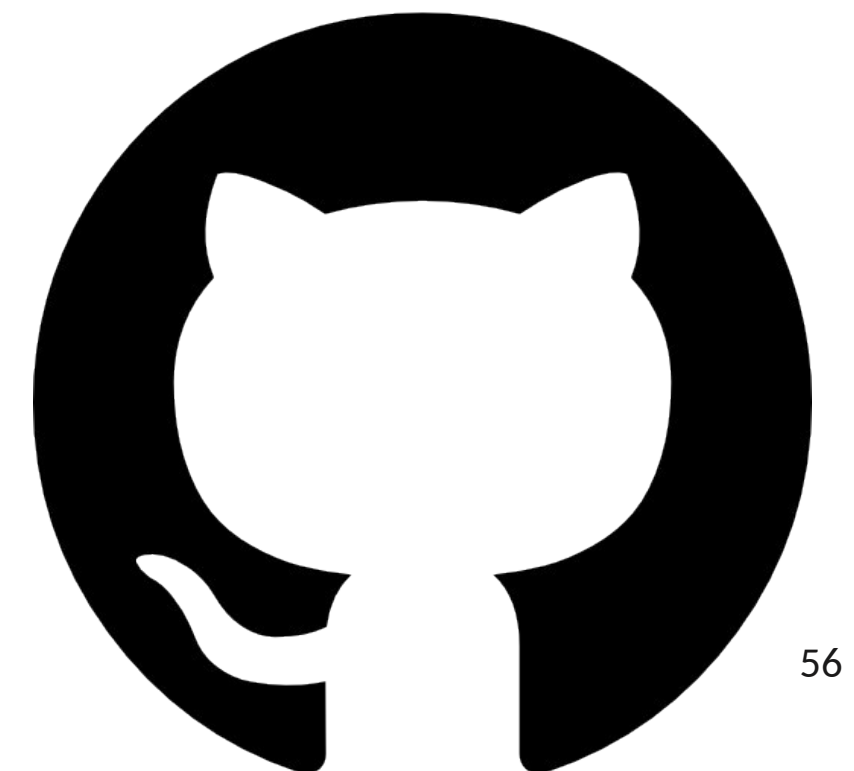
Используется, когда нужно объединить два массива поэлементно.

```
let names = ["Student1", "Student2", "Student3"]
let scores = [3, 5, 4]

let result = Array(zip(names, scores))
print(result)
// [("Student1", 3), ("Student2", 5), ("Student3", 4)]
```

Домашнее задание

- Текст задания найдете на edu
- Сделайте задание в Playground
- Сделайте ваши изменения в отдельной ветке
- PR в master (main) отправьте в форму





Спасибо!
Вопросы?

